

Làm Chủ Kỹ Năng In ấn trên Android

Từ một tấm ảnh đơn giản đến tài liệu tùy chỉnh hoàn toàn



Hướng dẫn toàn diện giúp bạn tích hợp chức năng in ấn mạnh mẽ vào ứng dụng của mình, nâng cao trải nghiệm người dùng và khả năng chia sẻ nội dung.

Tại sao In ấn lại Quan trọng trong Ứng dụng của Bạn?



Chia sẻ thông tin rộng rãi

Mang đến cho người dùng một cách để chia sẻ nội dung với những người không sử dụng ứng dụng của bạn.



Tạo bản sao vật lý

Cho phép người dùng xem một phiên bản lớn hơn của nội dung hoặc tạo một bản sao tổng quan nhanh.



Truy cập ngoại tuyến

Tạo ra thông tin không phụ thuộc vào việc có thiết bị, đủ pin hay kết nối mạng không dây.

Hành trình của Chúng ta: Ba Cấp độ In ấn



Phần 1 - Nhanh & Dễ dàng

In Ảnh: Sử dụng `PrintHelper` để in một hình ảnh với vài dòng mã.



Phần 2 - Linh hoạt & Có cấu trúc

In Nội dung HTML: Sử dụng `WebView` để in các trang có cấu trúc phức tạp hơn như báo cáo hoặc bài viết.



Phần 3 - Toàn quyền Sáng tạo

In Tài liệu Tùy chỉnh: Sử dụng `PrintDocumentAdapter` để kiểm soát tuyệt đối mọi chi tiết trên trang in.

Phần 1: In Ảnh - Cách Nhanh Nhất

PrintHelper 

Lớp `PrintHelper` từ Thư viện hỗ trợ Android (v4 support library) là giải pháp đơn giản nhất để in hình ảnh. Chức năng này cho phép in hình ảnh bằng số số lượng mã tối thiểu và một tập hợp các tùy chọn bố cục đơn giản.

Triển khai `PrintHelper` trong Thực tế

Các Chế độ Co giãn (Scale Modes)



`SCALE\_MODE\_FIT`: Định cỡ hình ảnh sao cho toàn bộ hình ảnh hiển thị trong vùng in được trên trang. (Mặc định)



`SCALE\_MODE\_FILL`: Điều chỉnh tỷ lệ hình ảnh để hình ảnh lấp đầy toàn bộ vùng in được trên trang.

Lưu ý: Cả hai tùy chọn đều giữ nguyên tỷ lệ khung hình hiện tại của hình ảnh.

Mã nguồn Kotlin

```
private fun doPhotoPrint() {  
    activity?.also { context ->  
        PrintHelper(context).apply {  
            scaleMode =                       
            PrintHelper.SCALE_MODE_FIT  
        }.also { printHelper ->  
            val bitmap =  
                BitmapFactory.decodeResource  
                    (resources, R.drawable.droids)  
            printHelper.printBitmap                       
                ("droids.jpg - test print", bitmap)  
        }  
    }  
}
```

Chọn chế độ co giãn

Bắt đầu lệnh in với ảnh bitmap

Phần 2: In Nội dung HTML với `WebView`

2



Để in nội dung ngoài một bức ảnh đơn giản, bạn phải soạn văn bản và đồ hoạ trong một tài liệu. Khung Android cung cấp cách sử dụng HTML để thực hiện việc này. Kể từ Android 4.4 (API cấp 19), lớp `WebView` cho phép bạn tải tài nguyên HTML cục bộ hoặc một trang web, tạo lệnh in và chuyển nó cho dịch vụ in của Android.

Quy trình In ấn với `WebView`

- 1 Tạo một `WebView`**
Có thể tạo đối tượng này riêng cho mục đích in, không cần hiển thị trên giao diện người dùng.
- 2 Thiết lập `WebViewClient`**
Ghi đè phương thức `onPageFinished()` để đảm bảo nội dung đã được tải hoàn toàn trước khi bắt đầu in.
- 3 Tải nội dung HTML**
Sử dụng `loadDataWithBaseURL()` cho chuỗi HTML hoặc `loadUrl()` cho một trang web.
- 4 Bắt đầu lệnh in**
Trong `onPageFinished()`, lấy `PrintManager` và gọi phương thức `print()` với `PrintDocumentAdapter` được tạo từ `WebView`.



Lưu ý quan trọng: Luôn gọi lệnh in bên trong `onPageFinished()`. Nếu không, bản in có thể bị trống, không hoàn chỉnh hoặc thất bại hoàn toàn.

Mã nguồn `WebView` trong Thực tế

```
private var mWebView: WebView? = null
```

```
private fun doWebViewPrint() {  
    val webView = WebView(activity)  
    webView.webViewClient = object : WebViewClient() {  
        override fun onPageFinished(view: WebView, url: String) {  
            createWebPrintJob(view)  
            mWebView = null  
        }  
    }  
    val htmlDocument = "<html><body><h1>Nội dung kiểm tra</h1></body></html>"  
    webView.loadDataWithBaseURL(null, htmlDocument, "text/HTML", "UTF-8", null)  
    mWebView = webView // Giữ tham chiếu để tránh bị garbage collector thu dọn  
}
```

```
private fun createWebPrintJob(webView: WebView) {  
    (activity?.getSystemService(Context.PRINT_SERVICE) as? PrintManager)?.let { printManager ->  
        val jobName = "${getString(R.string.app_name)} Document"  
        val printAdapter = webView.createPrintDocumentAdapter(jobName)  
        printManager.print(jobName, printAdapter, PrintAttributes.Builder().build())  
    }  
}
```

G Chỉ bắt đầu in sau khi trang đã tải xong.

G WebView tạo adapter in cho chúng ta.

G Quan trọng: Giữ tham chiếu để tránh WebView bị hủy sớm.

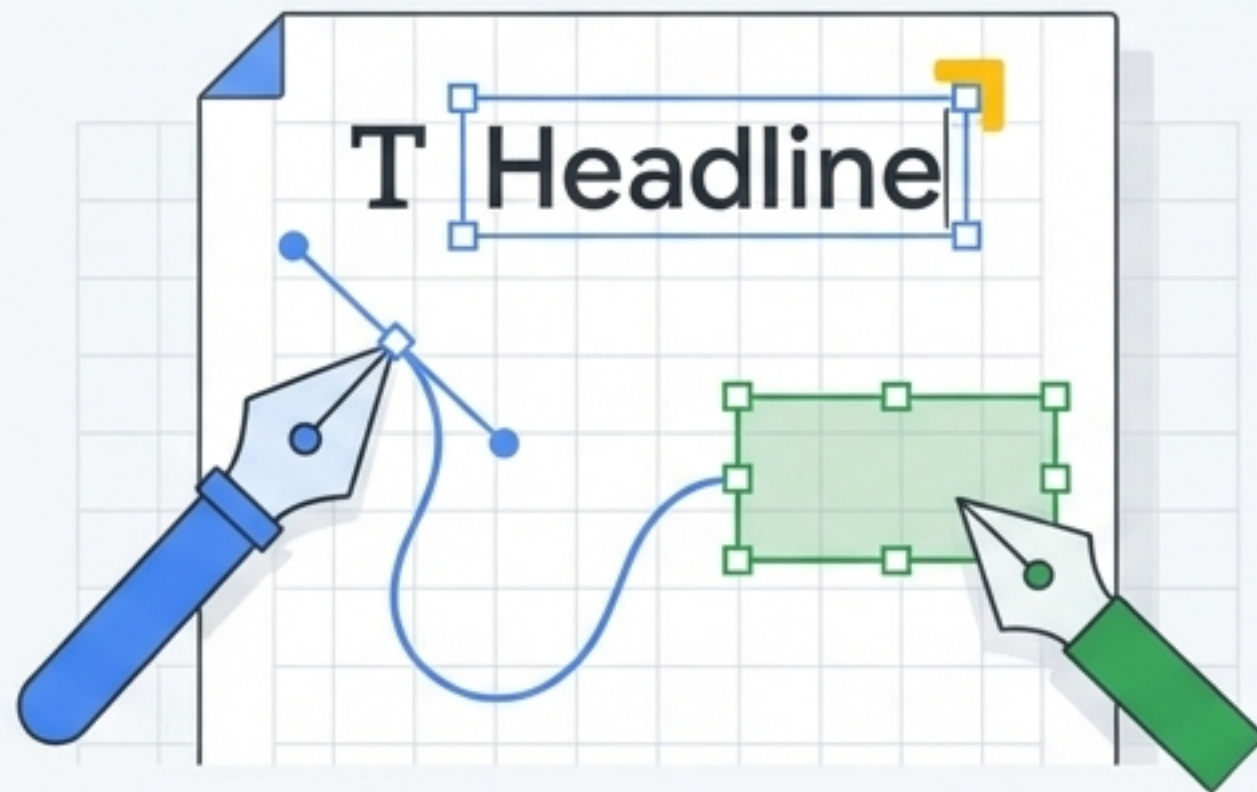


Những Hạn chế cần Biết của `WebView`

- ✗ Bạn không thể thêm đầu trang hoặc chân trang, bao gồm cả số trang.
- ✗ Tùy chọn in không bao gồm khả năng in theo khoảng trang (ví dụ: in trang 2 đến 4).
- ✗ Một thực thể của `WebView` chỉ có thể xử lý một lệnh in tại mỗi thời điểm.
- ✗ Bạn không thể sử dụng JavaScript trong tài liệu HTML để kích hoạt tính năng in.

3

Phần 3: Toàn quyền Sáng tạo với Tài liệu Tùy chỉnh



Đối với các ứng dụng vẽ, bố cục trang, hoặc các ứng dụng tập trung vào đồ họa, việc tạo ra các bản in đẹp là một tính năng quan trọng. Để kiểm soát chính xác mọi nội dung trên một trang—bao gồm phong chữ, dòng văn bản, ngắt trang, đầu trang, chân trang và các thành phần đồ họa—bạn cần xây dựng tài liệu in của riêng mình bằng cách triển khai lớp trừu tượng **PrintDocumentAdapter**.

Hiểu Vòng đời của `PrintDocumentAdapter`

`onStart()`

Được gọi một lần khi bắt đầu. Dùng cho các tác vụ chuẩn bị một lần.



`onLayout()`

Được gọi mỗi khi người dùng thay đổi cài đặt in (kích thước, hướng trang). Phải tính toán và trả về số lượng trang.



`onWrite()`

Được gọi để vẽ (render) nội dung các trang vào một tệp PDF. Có thể được gọi nhiều lần.



`onFinish()`

Được gọi một lần khi kết thúc. Dùng cho các tác vụ dọn dẹp.

Ghi chú: Các phương thức này được gọi trên luồng chính. Các tác vụ dài hơi nên được thực hiện trên một luồng riêng (ví dụ: `AsyncTask`).

Triển khai `onLayout()`: Tính toán Bố cục

- **Nhiệm vụ:** Phương thức này phải tính toán số lượng trang dự kiến dựa trên `PrintAttributes` (kích thước, hướng giấy) do hệ thống cung cấp.
- **Kết quả:** Thông tin này (số trang, loại nội dung) được trả về cho khung in thông qua một đối tượng **PrintDocumentInfo**.

```
override fun onLayout(oldAttributes: PrintAttributes?, newAttributes: PrintAttributes,
                    cancellationSignal: CancellationSignal?,
                    callback: LayoutResultCallback, extras: Bundle?) {
    // Phản hồi yêu cầu hủy
    if (cancellationSignal?.isCanceled == true) {
        callback.onLayoutCancelled()
        return
    }
    // Tính toán số trang dự kiến
    val pages = computePageCount(newAttributes)
    if (pages > 0) {
        val info = PrintDocumentInfo.Builder("print_output.pdf")
            .setContentType(PrintDocumentInfo.CONTENT_TYPE_DOCUMENT)
            .setPageCount(pages)
            .build()
        // Báo cho framework rằng việc tính toán bố cục đã hoàn tất
        callback.onLayoutFinished(info, true)
    } else {
        callback.onLayoutFailed("Page count calculation failed.")
    }
}
```

Xây dựng thông tin tài liệu: tên, loại nội dung, và số trang.

Báo cáo việc hoàn thành bố cục cho hệ thống.

Triển khai `onWrite()` : Ghi Nội dung ra Tập

Nhiệm vụ: Hiển thị từng trang nội dung được yêu cầu vào một tập tài liệu PDF nhiều trang. Hệ thống cung cấp các khoảng trang (pageRanges) và một `ParcelFileDescriptor` để ghi dữ liệu vào.

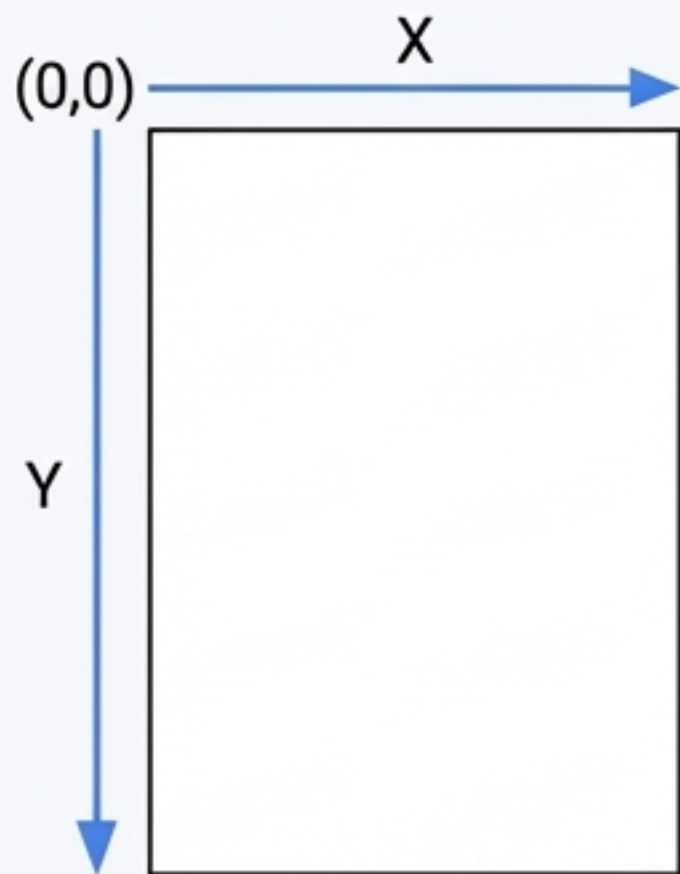
Quy trình: Lặp qua các trang, kiểm tra xem trang có nằm trong phạm vi yêu cầu không, sau đó vẽ nội dung cho trang đó.

```
override fun onWrite(pageRanges: Array<out PageRange>, destination: ParcelFileDescriptor,  
                    cancellationSignal: CancellationSignal?, callback: WriteResultCallback) {  
    for (i in 0 until totalPages) {  
        if (containsPage(pageRanges, i)) { // Hàm trợ giúp để kiểm tra  
            val page = pdfDocument.startPage(i)  
            // Kiểm tra yêu cầu hủy  
            if (cancellationSignal?.isCanceled == true) { ... return }  
            // Vẽ nội dung trang  
            drawPage(page)  
            pdfDocument.finishPage(page)  
        }  
    }  
    // Ghi tài liệu PDF vào tập  
    try {  
        pdfDocument.writeTo(FileOutputStream(destination.fileDescriptor))  
    } catch (e: IOException) {  
        callback.onWriteFailed(e.toString())  
        return  
    } finally {  
        pdfDocument.close()  
    }  
    // Báo cho framework rằng việc ghi đã hoàn tất  
    callback.onWriteFinished(writtenPages)  
}
```


Vẽ Nội dung lên Trang PDF bằng `Canvas`

Lớp `PrintedPdfDocument` sử dụng một đối tượng `Canvas` để vẽ các phần tử lên trang PDF, tương tự như việc vẽ trên một View tùy chỉnh.

Các đơn vị được tính bằng điểm (points), với 1 inch = 72 điểm. Gốc tọa độ (0,0) nằm ở góc trên cùng bên trái của trang.



```
private fun drawPage(page: PdfDocument.Page) {  
    page.canvas.apply {  
        val titleBaseLine = 72f  
        val leftMargin = 54f  
        val paint = Paint()  
        paint.color = Color.BLACK  
        paint.textSize = 36f  
        drawText("Tiêu đề Thử nghiệm", leftMargin, titleBaseLine, paint)  
  
        paint.textSize = 11f  
        drawText("Đoạn văn thử nghiệm...", leftMargin, titleBaseLine + 25, paint)  
    }  
}
```

Tiêu đề Thử nghiệm

Đoạn văn thử nghiệm...

Mẹo chuyên nghiệp: Nhiều máy in không thể in sát mép giấy. Hãy tính đến các lề không in được khi bạn thiết kế bố cục.

Chọn Công cụ Phù hợp cho Công việc

Tiêu chí	PrintHelper	WebView	PrintDocumentAdapter
Độ phức tạp	Thấp	Trung bình	Cao
Mức độ kiểm soát	Tối thiểu (chỉ có giãn ảnh)	Cơ bản (dựa trên bố cục HTML/CSS)	Toàn quyền (đồ họa, phong chữ, bố cục pixel-perfect)
Trường hợp sử dụng	In một ảnh duy nhất từ Bitmap hoặc Uri.	In nội dung web, báo cáo, hóa đơn có cấu trúc.	Ứng dụng vẽ, thiết kế bố cục, bản in tùy chỉnh hoàn toàn.